

CoInfoSim: simulation process for synthetic scenarios and real-data-based scenarios

Preliminary technical document

June 2026

Abstract

This document presents, in a preliminary technical format, the simulation process adopted in CoInfoSim, a simulator for studying cooperation among information channels in supervised classification tasks. The focus is on two types of scenario: (i) synthetic scenarios, in which data are generated from idealized probabilistic models, and (ii) real-data-based scenarios, in which training samples are extracted from a real dataset and evaluated on a fixed test set. The objective of the simulator is not merely to choose the subset of variables with the lowest error, but to produce diagnostics about dominance, redundancy, complementarity, and robustness of information channels as the number of training samples increases.

1 General motivation

CoInfoSim starts from a simple question: in a classification problem with multiple information channels, when does the combination of channels truly improve the decision? In many applications, especially in measurement systems, environmental monitoring, or sensor networks, the presence of multiple variables does not automatically imply a performance gain. Some channels may be dominant, others may be redundant, and others may look weak in isolation but become useful when combined with primary channels.

The purpose of the simulator is to evaluate this behavior systematically. Given a set of channels

$$X = (X_1, X_2, \dots, X_d)$$

and a class variable

$$Y \in \{1, \dots, K\},$$

CoInfoSim compares classifiers trained with different subsets of channels

$$A \subseteq \{1, \dots, d\}, \quad A \neq \emptyset.$$

For each subset, the classifier observes only

$$X_A = \{X_j : j \in A\}.$$

This formulation shifts the focus from a fixed incremental comparison, for example “one channel versus two channels” or “two channels versus three channels”, to a more general analysis of the available channel network. When $d = 5$, for example, the simulator evaluates

$$2^5 - 1 = 31$$

non-empty subsets. Thus, it allows us to observe not only whether adding channels reduces the average error, but which combinations are actually informative, which channels dominate the decision, and which groups appear to carry duplicated information.

Core idea

CoInfoSim estimates how classification error varies as a function of the number of training samples per class, the subset of channels used, and the type of classifier. The result is a learning curve for each subset, enabling diagnostics of cooperation, redundancy, and dominance among channels.

2 Common structure of the experiments

In both the synthetic scenario and the real-data-based scenario, the experimental core follows the same structure. For each sample size per class n , for each Monte Carlo replication r , for each channel subset A , and for each classifier f , the simulator trains a model and estimates the error on the test set.

The sequence can be summarized as

$$n \longrightarrow r \longrightarrow A \longrightarrow f.$$

The main sampling unit is

$$n = \text{number of training samples per class.}$$

In a binary problem, for example,

$$|D_0^{(r)}(n)| = |D_1^{(r)}(n)| = n,$$

and the balanced training set for replication r is

$$D_{\text{train}}^{(r)}(n) = D_0^{(r)}(n) \cup D_1^{(r)}(n).$$

Therefore, for K classes, training contains

$$Kn$$

samples in total.

The main metric is empirical loss on the fixed test set:

$$L_{A,f}^{(r)}(n) = \frac{1}{|D_{\text{test}}|} \sum_{(x_i, y_i) \in D_{\text{test}}} \mathbb{1}\{f_A^{(r,n)}(x_{i,A}) \neq y_i\}.$$

The Monte Carlo average is

$$\bar{L}_{A,f}(n) = \frac{1}{R_n} \sum_{r=1}^{R_n} L_{A,f}^{(r)}(n),$$

where R_n is the number of replications executed for sample size n .

2.1 Classifiers

The simulator can compare different classifier families. In the current experiments, three simple and interpretable classifiers were considered:

- linear SVM;
- logistic regression;
- Gaussian Naive Bayes.

The initial choice of simple classifiers is deliberate. The objective is to isolate the informational behavior of the channels, avoiding overly flexible models that could hide the contribution of subsets due to excessive fitting capacity.

2.2 Simulation budgets

Experiments are organized into execution modes. The purpose is to enable quick pipeline validation, intermediate experiments, and more complete runs.

Mode	Sample sizes n per class	Purpose
smoke	1, 2, 4, 8, 16, 32	quick pipeline test
fast	1, 2, 4, 8, 16, 32, 64, 128	intermediate experiment
full	1, 2, 4, 8, 16, 32, 64, 128, 256, 512	more stable analysis

In addition to the sample sizes, each mode defines parameters such as the minimum number of replications, the maximum number of replications, the replication batch size, and the tolerance for statistical uncertainty.

3 Synthetic scenarios

A synthetic scenario is an experiment in which data are artificially generated from a probabilistic model specified by the researcher. This type of scenario is useful when one wants to control the information structure: class separation, correlation among channels, noise, redundancy, or complementarity.

3.1 Generative model

In the simplest case, a class-conditional Gaussian model is assumed:

$$X | Y = c \sim \mathcal{N}(\mu_c, \Sigma_c),$$

where μ_c is the mean vector for class c and Σ_c is the covariance matrix for class c . The synthetic scenario is defined by

$$\mathcal{M} = \{(\mu_c, \Sigma_c)\}_{c=1}^K.$$

For a binary problem, we have

$$\mathcal{M} = \{(\mu_0, \Sigma_0), (\mu_1, \Sigma_1)\}.$$

This model makes it possible to create controlled situations. For example, one channel may have large mean separation between classes, another may have moderate separation, and a third may have low marginal separation but low correlation with the others. In that case, the third channel may be weak in isolation but useful in combination.

3.2 Data generation

For each Monte Carlo replication and each sample size n , the simulator generates class-balanced samples. In a binary problem:

$$D_0^{(r)}(n) \sim \mathcal{N}(\mu_0, \Sigma_0),$$

$$D_1^{(r)}(n) \sim \mathcal{N}(\mu_1, \Sigma_1),$$

with

$$|D_0^{(r)}(n)| = |D_1^{(r)}(n)| = n.$$

The synthetic test set is generated once and kept fixed across replications, reducing variability caused by changes in the test set. Thus, the variation observed in the curves is mainly due to training-set sampling and channel selection.

3.3 Stratification and training

Class stratification ensures that, for each n , the classifier receives the same number of examples from each class. This prevents the observed behavior from being dominated by sample imbalance. For each replication r , the simulator trains the classifier using only the columns corresponding to subset A .

Therefore, for each pair (A, f) , we obtain a curve

$$n \mapsto \bar{L}_{A,f}(n).$$

These curves answer questions such as:

- which channel is strongest in isolation?;
- when does a pair of channels outperform the best isolated channel?;
- when does the full set outperform the best smaller subset?;
- are there weak channels that help when combined?;
- are there redundant channels that barely change the loss?

3.4 What the synthetic scenario returns

A synthetic scenario may return diagnostics such as:

- ranking of subsets by classifier and sample size;
- mean error curves $\bar{L}_{A,f}(n)$;
- standard error for each curve point;
- number of replications executed until convergence or maximum budget;
- N^* thresholds indicating when one subset starts outperforming another;
- evidence of dominance, redundancy, or complementarity among channels.

The threshold N^* can be defined on the grid as the smallest observed value of n at which subset B outperforms subset A :

$$N_{\text{grid}}^* = \min\{n : \bar{L}_{B,f}(n) < \bar{L}_{A,f}(n)\}.$$

It is also possible to estimate an interpolated version when there is a sign change in the difference

$$\Delta(n) = \bar{L}_{A,f}(n) - \bar{L}_{B,f}(n).$$

If $\Delta(n_i)$ and $\Delta(n_{i+1})$ have opposite signs, then

$$\tilde{N}^* = n_i + \frac{0 - \Delta(n_i)}{\Delta(n_{i+1}) - \Delta(n_i)}(n_{i+1} - n_i).$$

4 Real-data-based scenarios

A real-data-based scenario uses an empirical dataset as the source of samples. The idea is to evaluate cooperation among channels in an observed problem, preserving real characteristics that may not be well represented by an idealized model.

In the current example, the scenario uses the *Occupancy Detection* dataset, whose objective is to predict room occupancy from environmental variables:

$$X = (\text{Temperature}, \text{Humidity}, \text{Light}, \text{CO2}, \text{HumidityRatio}).$$

The class is

$$Y = \text{Occupancy}.$$

4.1 Train-test split

The real scenario uses an explicit split between training pool and fixed test set:

$$D_{\text{pool}} = \text{datatraining.txt},$$

$$D_{\text{test}} = \text{datatest.txt} \cup \text{datatest2.txt}.$$

This split prevents the simulator from randomly choosing training and test sets at each replication, keeping the test set fixed and comparable across subsets, classifiers, and sample sizes.

4.2 Standardization

Standardization is learned only from the training pool. For each channel j , we compute

$$\bar{x}_j = \frac{1}{|D_{\text{pool}}|} \sum_{i \in D_{\text{pool}}} x_{ij},$$

$$s_j = \text{sd}_{D_{\text{pool}}}(X_j),$$

and apply

$$x_{ij}^{\text{std}} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

The same parameters are used to transform the test set. Therefore, the standardized test set does not need to have zero mean or unit standard deviation. If this occurs, it may indicate a shift between training and test, not necessarily an error.

4.3 Stratified sampling from the real pool

For each sample size n and replication r , the simulator draws class-balanced samples from the real pool:

$$D_c^{(r)}(n) \subset D_{\text{pool},c},$$

with

$$|D_c^{(r)}(n)| = n.$$

In the binary case:

$$D_{\text{train}}^{(r)}(n) = D_0^{(r)}(n) \cup D_1^{(r)}(n).$$

The test set remains fixed:

$$D_{\text{test}}.$$

This strategy answers the question: if we had only n examples per class for training, which channels would be most useful for classifying new cases observed in the test set?

4.4 Gaussian scenario anchored in real data

Between a purely synthetic scenario and a purely empirical scenario there is an intermediate alternative: the Gaussian scenario anchored in real data. In this case, a Gaussian model is estimated from the standardized pool:

$$\hat{\mu}_c = \frac{1}{|D_c|} \sum_{i \in D_c} x_i,$$

$$\hat{\Sigma}_c = \text{Cov}_{i \in D_c}(X).$$

Then a synthetic scenario is generated from

$$X \mid Y = c \sim \mathcal{N}(\hat{\mu}_c, \hat{\Sigma}_c).$$

This experimental arm answers an important question: how much of the cooperation structure observed in the real data can be reproduced by a Gaussian model that preserves class-wise means and covariances?

If the Gaussian scenario reproduces the channel ranking from the real experiment, this suggests that the average structure of separation and covariance explains much of the cooperation. If it does not, this may indicate regimes, asymmetries, tails, non-Gaussian dependencies, temporal effects, or rare cases that the idealized model does not capture.

5 Monte Carlo, uncertainty, and stopping criteria

The Monte Carlo method is used because the performance of a subset depends on the sampled training examples. With few examples per class, two different samples may produce substantially different classifiers. Therefore, for each n , the simulator repeats the process multiple times.

After R_n replications, the mean and standard error of the loss are computed:

$$SE_{A,f}(n) = \frac{s_{A,f}(n)}{\sqrt{R_n}},$$

where $s_{A,f}(n)$ is the sample standard deviation of the losses across replications.

One possible stopping criterion is to interrupt replications when all pairs (A, f) reach a confidence-interval half-width below a target τ :

$$\max_{A,f} 1.96 SE_{A,f}(n) < \tau.$$

In practice, this criterion is combined with a minimum and maximum budget:

- minimum number of replications, to avoid premature stopping;
- maximum number of replications, to control computational cost;
- batch size, to evaluate the stopping criterion in blocks;
- tolerance τ , which defines the desired precision.

If the simulation reaches the statistical criterion before the maximum budget, convergence is recorded. Otherwise, stopping by maximum budget is recorded. Both pieces of information are important: a curve may appear stable while still having high uncertainty for some subsets.

6 Interpreting the results

CoInfoSim should not be interpreted merely as a mechanism for finding the lowest error. Its value lies in the joint reading of rankings, curves, uncertainties, and thresholds. Typical diagnostics include the following.

6.1 Dominance

A channel is dominant when it consistently appears in the best subsets and when its use in isolation already produces low error. In the occupancy dataset, for example, the light channel may dominate classification if the presence of people is strongly associated with room lighting.

Dominance is useful, but it can also hide fragilities. If a global model depends too much on light, it may have difficulty in cases such as an occupied room with the lights off or an empty room with the lights on.

6.2 Redundancy

Two channels are redundant when their combination improves little over the best of them in isolation. This may occur when they carry similar physical or statistical information. In environmental data, for example, humidity and humidity ratio may share a relevant part of the information.

6.3 Complementarity

Complementarity occurs when a channel that appears weak in isolation substantially improves performance when combined with another channel. This is one of the most interesting behaviors, since it suggests real cooperation among channels.

6.4 Robustness and regimes

The analysis may also indicate the need to decompose the problem into regimes. If a dominant channel controls the global decision, the simulator can motivate derived experiments such as:

- running the scenario without the dominant channel;
- evaluating error conditioned on ranges of the dominant channel;
- training separate models for regimes, for example high light and low light;
- comparing the usefulness of environmental channels when light is no longer informative.

Thus, the simulator stops being only a ranking tool and becomes an instrument for experimental diagnosis.

7 Comparison between the two scenario types

Aspect	Synthetic scenario	Real-data-based scenario
Data origin	Generated by a specified probabilistic model	Sampled from an empirical dataset
Experimental control	High: means, covariances, noise, and correlations are defined	Lower: preserves irregularities and the real structure of the data
Main objective	Test idealized hypotheses about cooperation among channels	Diagnose behavior in an observed problem
Fixed test set	Synthetic, generated once from the model	Empirical, defined by files or a fixed partition
Strength	Causal clarity and parameter control	Realism and ability to reveal non-idealized regimes
Limitation	May oversimplify the problem	May contain bias, temporal shift, noise, and operational confounding

The combination of the two scenario types is especially useful. The synthetic scenario allows us to ask whether a given idealized structure is sufficient to explain the observed cooperation. The real scenario allows us to verify whether that explanation survives when confronted with empirical data.

8 Expected report output

A report generated by CoInfoSim should ideally present:

1. definition of the scenario and channels;
2. training and test protocol;
3. sample sizes per class;
4. Monte Carlo budget and stopping criterion;
5. list of classifiers;
6. number of evaluated subsets;
7. loss curves by classifier and subset;
8. rankings of the best subsets;
9. uncertainty estimates;
10. N^* thresholds when applicable;
11. interpretation of dominance, redundancy, and complementarity.

At the current stage, the reports may still be closer to a technical validation of the pipeline than to a final presentation. Even so, preliminary reports can already indicate whether the simulator is capturing relevant phenomena.

9 Conclusion

CoInfoSim proposes a systematic way of studying cooperation among information channels in supervised classification. In synthetic scenarios, it allows controlled hypotheses about separation, correlation, and redundancy to be tested. In real-data-based scenarios, it allows us to observe how channels behave in an empirical problem and which combinations are more useful as the number of training samples increases.

The main contribution of the simulator is not only to produce the best subset of channels, but to create a diagnostic process. This process can reveal dominant channels, redundant channels, complementary combinations, and possible regimes in which global models are no longer sufficient. In this sense, CoInfoSim can serve both as an experimental tool and as support for formulating hypotheses about multichannel classification systems.